

Manual de uso — com.calculator.Calculator

Calculadora que interpreta expresiones en forma de tupla y devuelve el resultado. Soporta operaciones escalares (binarias y unarias), ecuaciones polinómicas, listas de primos, aritmética de polinomios y aritmética de matrices.

1. Compilar y ejecutar

Desde C:\javatest:

```
javac com\calculator\Calculator.java
java com.calculator.Calculator
```

Sin argumentos ejecuta una batería de ejemplos.

Con argumentos evalúa cada uno:

```
java com.calculator.Calculator "(4,2, /)" "[[(1,2),(3,4)],D]"
```

Nota: en PowerShell/Bash conviene poner comillas alrededor de cada expresión para evitar que la shell interprete los paréntesis o corchetes.

2. Formato general de entrada

La calculadora reconoce **cuatro formatos** según el carácter inicial:

Empieza por	Forma	Uso
(	(arg, arg, ..., OP)	operaciones escalares y polinomios (con X/Y, P)
[	[operando, operando, OP]	polinomios (+,-,*), matrices, escalar×matriz
[	[operando, OP]	operadores unarios sobre matriz: T, D
{	{n1, n2, ..., nk}	suma directa de todos los números

En todos los casos los espacios se ignoran y el **último token** es el operador.

Números aceptados

Token	Valor
123, -4.5, 1e3	Literal double
E o e	Math.E (2.71828...)
PI o pi	Math.PI (3.14159...)

3. Operaciones binarias escalares (a, b, OP)

OP	Nombre	Ejemplo	Resultado
+	Suma	(1, 1, +)	2
-	Resta	(7, 5, -)	2
*	Producto	(6, 3, *)	18
/	División	(4, 2, /)	2
^	Potencia	(3, 2, ^)	9
R	Raíz enésima (b = índice)	(9, 2, R)	3
L	Logaritmo (b = base)	(100, 10, L)	2

Casos especiales:

- (2, E, *) → $2 \cdot e$
- (PI, 3, *) → 3π
- (10, E, L) → logaritmo neperiano de 10
- (-8, 3, R) → -2 (raíz cúbica de -8)

Errores

- (1, 0, /) → ArithmeticException: División por cero.
- (-9, 2, R) → raíz de índice par de negativo.
- (1, 0, R) → índice 0.
- (0, 10, L) o base ≤ 0 o base = 1 → logaritmo inválido.

4. Operaciones unarias escalares (a, OP)

OP	Nombre	Ejemplo	Resultado
!	Factorial	(5, !)	120
F	Fibonacci n-ésimo ($F_0=0$, $F_1=1$)	(10, F)	55

Requieren entero ≥ 0 . ! está limitado a 170 (overflow en double).

5. Ecuaciones polinómicas (a_n, ..., a_0, X)

El operador X (o Y, equivalente) resuelve el polinomio con coeficientes de **mayor a menor grado**.

Entrada	Ecuación	Salida
(1, -3, 2, X)	$x^2 - 3x + 2 = 0$	X1 = 2, X2 = 1
(2, 2, 2, X)	$2x^2 + 2x + 2 = 0$	X1 = -0.5 + 0.866i, X2 = -0.5 - 0.866i
(1, -6, 11, -6, X)	$x^3 - 6x^2 + 11x - 6 = 0$	X1 = 1, X2 = 3, X3 = 2

- Grado 1: despeje directo.
- Grado 2: fórmula cuadrática (raíces complejas si el discriminante es negativo).
- Grado ≥ 3 : **Durand-Kerner** (raíces reales y complejas).
- El coeficiente principal no puede ser 0.

6. Números primos (..., P)

Entrada	Significado	Resultado
(10, P)	Primos ≤ 10	2, 3, 5, 7
(1, 100, P)	Primos en el intervalo [1, 100]	2, 3, 5, 7, ..., 97
(50, 70, P)	Primos en [50, 70]	53, 59, 61, 67

- Implementado con criba de Eratóstenes.
- **Límite superior:** 100 000 000 (100 M). Valores mayores lanzan excepción para evitar consumo excesivo de memoria.

7. Aritmética de polinomios [(p), (q), OP]

Los polinomios se representan como tuplas de coeficientes de **mayor a menor grado**. Operadores: +, -, *, .

Entrada	Cálculo	Resultado
[(1, 1), (1, -1), *]	$(x+1)(x-1)$	(1, 0, -1)
[(1, 1), (1, 1), *]	$(x+1)^2$	(1, 2, 1)
[(1, 2, 3), (4, 5, 6), +]	suma coef. a coef.	(5, 7, 9)
[(1, 2, 3), (1, 1), -]	$(x^2+2x+3)-(x+1)$	(1, 1, 2)
[(1, 0, 0, 0), (1, 0, 0, 0), *]	$x^3 \cdot x^3$	(1, 0, 0, 0, 0, 0, 0)

- Suma/resta alinean por el término independiente (rellenan a la izquierda con ceros si los grados difieren).
- Los ceros a la izquierda del resultado se eliminan.
- La división de polinomios **no está implementada**.

8. Aritmética de matrices [[m1], [m2], OP]

Una matriz se escribe con **corchetes exteriores** y una tupla (...) por fila. Los operadores son +, -, * (producto matricial).

[[(1, 1), (1, 1)], [(1, 1), (1, 1)], +]	→ [(2, 2), (2, 2)]
[[(1, 2), (3, 4)], [(5, 6), (7, 8)], -]	→ [(-4, -4), (-4, -4)]
[[(1, 2), (3, 4)], [(5, 6), (7, 8)], *]	→ [(19, 22), (43, 50)]

`[[ (1, 2, 3) ], [ (4), (5), (6) ], *]` $\rightarrow$ `[(32)]`

Vectores

- **Vector fila:** `[(a, b, c)]` (matriz 1×n).
- **Vector columna:** `[(a), (b), (c)]` (matriz n×1).
- El producto vector fila × vector columna devuelve una matriz 1×1 (escalar embebido).

Reglas

- `+/-`: exigen dimensiones idénticas.
- `*`: exige `cols(A) == rows(B)`. Resultado: `rows(A) × cols(B)`.
- Filas de longitud inconsistente $\rightarrow$ `IllegalArgumentException`.

9. Escalar × matriz `[k, [m], *]` o `[[m], k, *]`

El único operador admitido es `*`. El orden es indiferente.

`[3, [(1, 2), (3, 4)], *]` $\rightarrow$ `[(3, 6), (9, 12)]`
`[[[(1, 2), (3, 4)], 2], *]` $\rightarrow$ `[(2, 4), (6, 8)]`
`[E, [(1, 2)], *]` $\rightarrow$ `[(2.7182..., 5.4365...)]`

10. Traspuesta `[[m], T]`

Formato unario. La fila i pasa a ser la columna i.

`[[[(1, 2, 3), (4, 5, 6)], T]` $\rightarrow$ `[(1, 4), (2, 5), (3, 6)]`
`[[[(1), (2), (3)], T]` $\rightarrow$ `[(1, 2, 3)]` (columna $\rightarrow$ fila)

11. Determinante `[[m], D]`

Formato unario. Devuelve un escalar. La matriz debe ser cuadrada.

`[[[(1, 2), (3, 4)], D]` $\rightarrow$ `-2`
`[[[(6, 1, 1), (4, -2, 5), (2, 8, 7)], D]` $\rightarrow$ `-306`

- Algoritmo: eliminación gaussiana con pivoteo parcial (complejidad $O(n^3)$).
- Matriz no cuadrada $\rightarrow$ `IllegalArgumentException` con las dimensiones.

12. Resumen de operadores

Escalares (...)

OP	Aridad	Descripción
<code>+</code>	binario	suma
<code>-</code>	binario	resta
<code>*</code>	binario	producto
<code>/</code>	binario	división

OP	Aridad	Descripción
<code>^</code>	binario	potencia
<code>R</code>	binario	raíz de índice b
<code>L</code>	binario	logaritmo base b
<code>!</code>	unario	factorial
<code>F</code>	unario	n-ésimo Fibonacci
<code>P</code>	1 o 2 args	primos $\leq n$ o en $[a, b]$
<code>X</code>	≥ 2 args	raíces del polinomio
<code>Y</code>	≥ 2 args	alias de X

Estructuras [...]

OP	Aridad	Operandos	Descripción
<code>+</code>	binario	2 polinomios / 2 matrices	suma
<code>-</code>	binario	2 polinomios / 2 matrices	resta
<code>*</code>	binario	2 polinomios / 2 matrices	producto (convolución / matricial)
<code>*</code>	binario	escalar + matriz	producto escalar por matriz
<code>T</code>	unario	1 matriz	traspuesta
<code>D</code>	unario	1 matriz cuadrada	determinante

13. API pública

```
double d          = Calculator.evaluate("(4,2,/");           // 2.0
Complex[] roots   = Calculator.solve("(1,-3,2,X)");          // {2, 1}
String texto      = Calculator.process("[[(1,2),(3,4)],D]");  // "-2"
```

- `evaluate(String)` → `double`. Solo para operaciones escalares (binarias y unarias). Lanza `IllegalStateException` para entradas polinómicas o estructurales.
- `solve(String)` → `Calculator.Complex[]`. Solo para ecuaciones polinómicas `X/Y`.
- `process(String)` → `String`. **Dispatcher universal**: elige el modo adecuado y formatea el resultado como texto.

`Calculator.Complex` expone campos públicos `re` e `im` y un `toString()` legible.

14. Manejo de errores

Todas las excepciones son subclases de `RuntimeException`:

- `IllegalArgumentException` — formato inválido, operador desconocido, número no parseable, dimensiones incompatibles, matriz mal formada.
- `ArithmeticException` — división por cero, raíz par de negativo, logaritmo con argumento/base inválidos, factorial/Fibonacci/primos no entero o negativo, factorial > 170, primos con límite > 10^8 .
- `IllegalStateException` — llamar a `evaluate()` con una entrada polinómica.

El `main` incorpora `try/catch` que imprime `ERROR: <mensaje>` en lugar de terminar el proceso.

15. Suma de array $\{n_1, n_2, \dots, n_k\}$

Formato para sumar una lista arbitraria de números. Disponible tanto en la versión completa como en la reducida.

Entrada	Resultado
<code>{1, 2, 3, 4, 5}</code>	15
<code>{PI, E}</code>	$\pi + e$
<code>{10, -3, 2.5}</code>	9.5
<code>{}</code>	0

Acepta literales numéricos y las constantes `E/PI`. Errores: falta `}` de cierre → `IllegalArgumentException`.

16. Versión reducida

`CalculatorFree.java` es un archivo autónomo que ofrece únicamente `+`, `-`, `*`, `/` sobre operandos `(a, b, op)` (con `E/PI` como constantes) y la suma de array $\{n_1, \dots, n_k\}$. Se compila y ejecuta igual:

```
javac com\calculator\CalculatorFree.java
java com.calculator.CalculatorFree "(2,E,*)"
```